Universidad Nacional Rosario Castellanos

Ciencia de Datos para Negocios

Agroasis: Sistema de Recomendación de Cultivos y Optimización de Fertilizante en Parcelas Agrícolas

Problema prototípico presentado por:	***Aquino Soto Joshua David***
	Gervacio Peralta Carlos David
	Romero Morales Ángel Jerath
Tipo de trabajo:	*Agroasis: Sistema de recomendación de cultivos*
	y optimización de fertilizante
	en parcelas agrícolas
Profesor/a:	*Barragán Trinidad Melesio*
	Castillo Vieyra Ana Karen
	Hernández Jiménez Sharon
Tutor/a:	*Sierra Hernández María Guadalupe*
	Hernández Jiménez Sharon

Resumen

Agroasis es un sistema informático que analiza datos de suelo (nitrógeno, fósforo, potasio, pH y materia orgánica) para recomendar cultivos y dosificar fertilizantes de manera óptima. Mediante un modelo de Random Forest clasifica la fertilidad del suelo y, a partir de álgebra vectorial y cálculo diferencial, determina la cantidad exacta de fertilizante necesaria para cada parcela. El sistema convierte automáticamente las unidades de laboratorio a escalas agronómicas, acepta múples muestras por parcela y proyecta la degradación del suelo durante cinco temporadas si no se reponen nutrientes. Integra además una interfaz visual con semáforo de salud del suelo y costos estimados de insumos, facilitando decisiones sostenibles y rentables en el campo.

Palabras clave: agricultura de precisión, fertilización, Random Forest, análisis de suelos, sostenibilidad agrícola.

Abstract

Agroasis is a software system that analyzes soil data (nitrogen, phosphorus, potassium, pH, and organic matter) to recommend crops and determine optimal fertilizer doses. Using a Random Forest classifier, vector algebra, and differential calculus, it identifies the best-fitting crops and the exact fertilizer amount each plot needs. The system automatically converts laboratory units to agronomic scales, handles multiple samples per plot, and simulates soil nutrient depletion over five growing seasons without replenishment. A visual interface with a soil health traffic-light indicator and estimated input costs supports practical, data-driven decision-making in the field.

Keywords: precision agriculture, fertilization, Random Forest, soil analysis, agricultural sustainability.

Índice

1. Introducción **5**

1.1. Introducción de la problemática 5

1.2. Planteamiento del problema 5

1.2.1. Incidentes críticos 5

1.3. Objetivos del problema prototípico 6

1.3.1. Objetivo general 6

1.3.2. Objetivos específicos 6

2. Propuestas de solución **7**

2.1. Unidades de medida 7

2.1.1. Fórmula de conversión 7

2.1.2. Qué valores se convierten y cuáles no 8

2.1.3. Dónde ocurre la conversión en el código 8

2.2. Random Forest 9

2.2.1. Preparación del conjunto de datos 9

2.2.2. Entrenamiento del modelo 10

2.3. Predicción con probabilidades 11

2.3.1. Importancia de variables 12

2.4. Álgebra lineal 12

2.4.1. Vectorización 12

2.4.2. Vector de demanda del cultivo 12

2.4.3. Vector de aporte del suelo 13

2.4.4. Operación: Resta de vectores 13

2.4.5. Clasificación del estado por umbrales agronómicos 14

2.5. Selección del fertilizante 15

2.5.1. Algoritmo greedy 15

2.6. Cálculo diferencial 16

2.6.1. Optimización de dosis de fertilizante 16

2.6.2. El cálculo del punto óptimo 17

2.6.3. Parametrización dinámica 18

2.6.4. Lectura de la gráfica 18

2.7. Estadística 19

2.7.1. Promedio de múltiples muestras por parcela 19

2.7.2. Identificación automática de parcelas . . . 19
2.7.3. Cálculo de media aritmética . . . 20
2.7.4. Estadísticas descriptivas adicionales . . . 21
2.7.5. Prueba de hipótesis para desequilibrios nutricionales . . . 22
2.8. Evolución del suelo sin reposición . . . 23
2.8.1. El problema de la sostenibilidad . . . 23
2.8.2. Inferencia para casos sin datos . . . 23

3. Marco teórico **24**
3.1. Agronomía y propiedades físico-químicas del suelo . . . 24
3.1.1. Nutrición vegetal . . . 24
3.1.2. pH y conductividad eléctrica . . . 25
3.2. Agricultura de precisión . . . 25
3.3. Machine learning . . . 25
3.4. Fundamentos matemáticos . . . 26
3.4.1. Modelos de predicción y reconocimiento de patrones . . . 26
3.4.2. Similitud de datos . . . 26
3.4.3. Sistemas de apoyo para la toma de decisiones agrícolas . . . 26

4. Limitaciones y prospectivas **27**
4.1. Limitaciones . . . 27
4.1.1. La limitante de datos agronómicos en México . . . 27
4.2. Metodología de construcción de la base de datos . . . 27
4.2.1. Cambio de escala (mg/kg a kg/ha) . . . 29
4.2.2. Estructura final del dataset . . . 30
4.3. Prospectivas . . . 30

5. Conclusión **32**

Índice de figuras

1. Dinámica de optimización: intersección entre el rendimiento marginal decreciente y la degradación cuadrática. 19

Índice de cuadros

1. Conversión de parámetros y unidades del modelo. 8
2. Nivel de confianza agrícola según el tipo de cultivo. 11
3. Ejemplo completo con valores reales de requerimientos nutricionales (jitomate). . . 14
4. Resultados de las muestras individuales y cálculo del promedio. 21
5. Conversión del vector promedio a unidades de campo. 21
6. Mapeo del *score* de salud a estados visuales. 22
7. Referencias técnicas del INIFAP utilizadas como valores ancla. 28

1. Introducción

1.1. Introducción de la problemática

En México, según datos del Servicio de Información Agroalimentaria y Pesquera (Servicio de Información Agroalimentaria y Pesquera, 2022), el 44.9 % del territorio nacional presenta algún nivel de deterioro edáfico. Además, *"aproximadamente el 62 % de los suelos agrícolas muestra degradación física o química"* (Food and Agriculture Organization of the United Nations, 2015), derivada en buena parte del uso inadecuado de fertilizantes. Una gran proporción de los productores aplica estos insumos sin un diagnóstico previo del suelo, lo que genera sobrefertilización, pérdida de nitrógeno hacia los acuíferos y costos innecesarios de producción (Obalum et al., 2017). Dado que *"los fertilizantes representan entre el 30 y el 50 por ciento de los costos variables en la producción agrícola"* (Secretaría de Agricultura y Desarrollo Rural, 2020), el impacto económico de una dosificación sin respaldo técnico es significativo.

Agroasis es un sistema informático construido en Python que recibe el análisis de suelo de una parcela en las unidades que reporta cualquier laboratorio comercial (mg/kg) y entrega recomendaciones sobre la selección de cultivos y la dosis óptima de fertilizante. Para ello integra un clasificador de Random Forest (Breiman, 2001) entrenado con perfiles de suelo del contexto mexicano, álgebra vectorial para el cálculo de déficits nutricionales y cálculo diferencial para encontrar el punto óptimo de aplicación. Cuenta también con un módulo que proyecta el agotamiento del suelo durante cinco temporadas sin reposición de nutrientes, apoyando la planificación a mediano plazo.

1.2. Planteamiento del problema

1.2.1. Incidentes críticos

La fertilidad del suelo es la base de la productividad agrícola y su conservación es un reto prioritario en el campo mexicano. *"El 62 % de los suelos en México presenta degradación física o química y el 44.9 % del territorio nacional muestra algún nivel de deterioro"* (Food and Agriculture Organization of the United Nations, 2015). Esta situación evidencia la urgencia de herramientas que favorezcan el análisis y seguimiento del estado del suelo.

La ausencia de trazabilidad histórica en los análisis de suelos dificulta identificar cambios a lo largo del tiempo y genera decisiones empíricas que ocasionan aplicaciones inadecuadas de fertilizantes, aumento de costos y afectaciones al ecosistema. Organizar, clasificar y contrastar la información disponible se vuelve indispensable para orientar el manejo del suelo con mayor

precisión técnica.

Sustituir criterios intuitivos por decisiones basadas en evidencia es necesario, considerando que muchos productores aplican fertilizantes sin diagnóstico previo (Secretaría de Agricultura y Desarrollo Rural, 2020). Una dosificación incorrecta puede provocar pérdida de nitrógeno y afectar la calidad de los acuíferos (Obalum et al., 2017), además de reducir la rentabilidad del productor. De aquí surge la necesidad de un sistema que analice los datos del suelo y los traduzca en recomendaciones claras y accionables.

1.3. Objetivos del problema prototípico

1.3.1. Objetivo general

Construir el sistema informático Agroasis utilizando el lenguaje de programación Python para la categorización y administración avanzada de la salud del suelo en México, aplicando algoritmos de análisis de datos que permitan maximizar el uso de fertilizantes, disminuir los gastos de producción y promover una transición hacia métodos agrícolas sostenibles fundamentados en datos científicos.

1.3.2. Objetivos específicos

Organizar la recopilación y el procesamiento de un dataset de suelo adaptado al contexto mexicano, utilizando métodos de depuración y normalización de datos dentro de una estructura orientada a objetos para asegurar la trazabilidad histórica y la coherencia de la información analizada.

Desarrollar un modelo de clasificación supervisada que facilite la categorización de los niveles de macronutrientes (nitrógeno, fósforo y potasio) y verificar su eficacia mediante métricas de validación técnica para proporcionar diagnósticos precisos que reemplacen las decisiones intuitivas con predicciones fidedignas.

Crear una interfaz práctica basada en un sistema de semaforización visual (rojo, naranja, verde) que simplifique la entrada de muestras para generar recomendaciones útiles sobre la elección de cultivos y fertilización, logrando optimizar el proceso de toma de decisiones y aumentar la rentabilidad del agricultor.

2. Propuestas de solución

2.1. Unidades de medida

El primer aspecto que el sistema resuelve de forma transparente es la conversión de unidades. Tanto el laboratorio como el dataset de entrenamiento manejan los nutrientes en **mg/kg**, pero los cálculos internos de dosificación requieren trabajar en **kg/ha** porque las plantas consumen masa, no concentración.

El **laboratorio** reporta concentraciones de nutrientes en **mg/kg** de suelo seco (partes por millón). Esta unidad mide cuántos miligramos de nutriente hay por kilogramo de suelo.

El **dataset de entrenamiento** también almacena los valores en **mg/kg**, tal como salen del laboratorio. Sin embargo, los cálculos internos del sistema requieren trabajar en **kg/ha** (kilogramos por hectárea) porque las plantas consumen *masa*, no concentración: es imposible calcular cuántos costales de fertilizante necesita una hectárea usando solo mg/kg.

Por eso Agroasis realiza una conversión automática e invisible: el usuario sube su CSV en mg/kg y el sistema transforma esos valores a kg/ha justo antes de alimentar al modelo o calcular déficits. Si se omitiera esta conversión, el sistema compararía escalas incompatibles: 45 mg/kg de nitrógeno del laboratorio contra una demanda de 180 kg/ha del cultivo.

2.1.1. Fórmula de conversión

La conversión se deriva de la física del suelo. Para calcular cuántos kilogramos de nutriente hay en una hectárea de suelo, se necesita conocer:

$$kg/ha = mg/kg \times DA \times Prof \times 0{,}1$$

Donde:

- DA = Densidad aparente del suelo (g/cm^3). Para suelo agrícola típico: $1{,}0\ g/cm^3$.
- $Prof$ = Profundidad de muestreo (cm). Estándar agroquímico: **20 cm** (zona radicular).
- $0{,}1$ = Factor de conversión dimensional (unifica g/cm^3, cm y mg/kg en kg/ha).

```
kg/ha = mg/kg * 1.0 * 20 * 0.1
      = mg/kg * 2.0
```

Este es el factor 2.0 que aparece en todo el código. Se considera una simplificación válida para el 95 % de los suelos agrícolas de México trabajados con labranza convencional a 20 cm de

profundidad. Es el mismo factor que emplea el INIFAP en sus tablas de interpretación de análisis de suelos (Instituto Nacional de Investigaciones Forestales, Agrícolas y Pecuarias, 2019).

2.1.2. Qué valores se convierten y cuáles no

No todos los parámetros del CSV subido por el usuario necesitan conversión.

Cuadro 1: Conversión de parámetros y unidades del modelo.

Parámetro	Unidad en CSV	¿Se convierte?	Unidad en el modelo
N_mgkg	mg/kg	Sí ($\times 2{,}0$)	kg/ha
P_mgkg	mg/kg	Sí ($\times 2{,}0$)	kg/ha
K_mgkg	mg/kg	Sí ($\times 2{,}0$)	kg/ha
ph	adimensional	No	adimensional
mo_pct	%	No	%
Ca_mgkg, Mg_mgkg...	mg/kg	No*	mg/kg

Los nutrientes como Ca, Mg y los micronutrientes siempre se usaron como variables de entrada al clasificador en mg/kg, porque el dataset de entrenamiento los mantuvo en esa escala. Solo N, P y K se convierten porque son los que se comparan contra demandas de cultivo expresadas en kg/ha.

2.1.3. Dónde ocurre la conversión en el código

La conversión se aplica en **dos lugares distintos** del sistema, por razones diferentes.

Lugar 1: Entrada al clasificador Random Forest

Cuando el usuario sube el CSV, el vector de entrada al clasificador convierte N, P y K antes de escalarlos con StandardScaler. Esto es necesario porque el modelo fue entrenado con esos valores ya convertidos:

```
# models/crop_classifier.py  metodo preparar_entrada()

X = []
for feature in SOIL_FEATURES:
    val = float(datos_dict.get(feature, 0))
    # N, P, K se convierten de mg/kg a kg/ha antes de entrar al modelo
    if feature in ["N_mgkg", "P_mgkg", "K_mgkg"]:
        val = val * 2.0
    X.append(val)
X = np.array(X).reshape(1, -1)
```

```
X_scaled = self.scaler.transform(X)   # Normalizacion con misma escala del
    entrenamiento
```

Lugar 2: Construcción del vector de aporte del suelo para el análisis de déficit

```
# models/fertilizer_recommender_v2.py  calcular_deficit()

n_suelo  = float(propiedades_suelo.get("N_mgkg", 0)) * 2.0  # mg/kg a kg/ha
p_suelo  = float(propiedades_suelo.get("P_mgkg", 0)) * 2.0
k_suelo  = float(propiedades_suelo.get("K_mgkg", 0)) * 2.0
ph_suelo = float(propiedades_suelo.get("ph",    7.0))       # sin conversion
mo_suelo = float(propiedades_suelo.get("mo_pct", 2.0))      # sin conversion
```

2.2. Random Forest

Un árbol de decisión simple divide el espacio de datos con reglas del tipo "si N >80 y pH <7, entonces maíz". Es interpretable, pero inestable: si se cambia una muestra, el árbol puede verse completamente distinto.

Random Forest construye 200 árboles en paralelo, cada uno entrenado sobre una muestra aleatoria del dataset y usando solo un subconjunto aleatorio de variables en cada nodo. La predicción final es la votación de los **200 árboles**. Esta técnica, denominada *bagging*, reduce la varianza sin aumentar el riesgo de sesgo, y es especialmente útil cuando las variables tienen escalas muy distintas entre sí, como ocurre en este caso: el pH va de 3 a 10, el calcio de 500 a 5000, (Breiman, 2001).

2.2.1. Preparación del conjunto de datos

Antes de entrenar, el sistema aplica tres transformaciones:

a) Codificación de textura

La textura del suelo ("franco", "arcilloso", "arenoso". . .) es categórica. Random Forest necesita números, no palabras:

```
# models/crop_classifier.py  entrenar()

self.encoder_textura = LabelEncoder()
df_train["textura_cod"] = self.encoder_textura.fit_transform(
    df_train["textura_suelo"]
)
# Resultado: "arenoso"=0, "arcilloso"=1, "franco"=2, etc.
```

b) Normalización (StandardScaler)

Con variables en escalas tan distintas, sin normalizar, el pH (3–10) tendría un peso ridículamente menor que el calcio (500–5000) por pura magnitud. $StandardScaler$ centra cada variable en media 0 y desviación estándar 1:

$$X_{norm} = \frac{(X - \mu)}{\sigma}$$

```
self.scaler = StandardScaler()
X_scaled = self.scaler.fit_transform(X)
# mu y sigma se calculan sobre el entrenamiento y se reutilizan en prediccion
```

c) Partición entrenamiento/prueba (80/20)

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y,
    test_size=0.2,
    random_state=42    # semilla fija para reproducibilidad
)
```

2.2.2. Entrenamiento del modelo

```
# models/crop_classifier.py

self.modelo = RandomForestClassifier(
    n_estimators=200,  # 200 arboles en el bosque
    random_state=42,   # resultado reproducible entre ejecuciones
    n_jobs=-1,         # paralelismo total: usa todos los nucleos del CPU
)
self.modelo.fit(X_train, y_train)

y_pred   = self.modelo.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
```

El modelo entrenado y el $scaler$ se serializan con `pickle` para no volver a entrenar en cada inicio del servidor:

```
with open(CROP_CLASSIFIER_PATH, 'wb') as f:
    pickle.dump(self.modelo, f)
with open(SCALER_PATH, 'wb') as f:
```

```
    pickle.dump(self.scaler, f)
```

2.3. Predicción con probabilidades

Para una nueva muestra de suelo (ya convertida a kg/ha y normalizada), el modelo devuelve la **probabilidad** promedio de los 200 árboles para cada cultivo del catálogo:

```
# models/crop_classifier.py  predecir_cultivos()

probabilidades = self.modelo.predict_proba(X_scaled)[0]
cultivos       = self.modelo.classes_

# Ordenar de mayor a menor probabilidad y tomar el top N
indices_top = np.argsort(probabilidades)[-n_mejores:][::-1]

resultados = []
for idx in indices_top:
    resultados.append({
        "cultivo":    cultivos[idx],
        "confianza":  float(probabilidades[idx]),
        "porcentaje": float(probabilidades[idx] * 100)
    })
```

Por ejemplo, para un suelo con las siguientes mediciones de laboratorio:
$N_{mg/kg} = 55 \rightarrow 110_{kg/ha}$ (después del factor ×2.0)
$K_{mg/kg} = 180 \rightarrow 360_{kg/ha}$
$pH = 6{,}8$ (sin conversión)
Textura: franco

Cuadro 2: Nivel de confianza agrícola según el tipo de cultivo.

Cultivo	**Confianza**
Maíz grano blanco	72.3 %
Frijol negro	14.1 %
Tomate rojo	8.5 %

El usuario especifica $N = 55\ mg/kg$ en su CSV. El sistema convierte internamente a 110 kg/ha antes de consultarle al modelo. El modelo nunca ve mg/kg directamente; opera siempre en la escala en que fue entrenado.

2.3.1. Importancia de variables

Random Forest calcula qué variables contribuyeron más a las decisiones del bosque completo:

```
importancias = self.modelo.feature_importances_
for feature, imp in sorted(zip(features, importancias),
                           key=lambda x: x[1], reverse=True):
    print(f"{feature}: {imp*100:.1f}%")
```

Típicamente, pH, $N_{mg/kg}$ y $K_{mg/kg}$ resultan ser las variables más determinantes para clasificar cultivos en el contexto mexicano, lo que es coherente con la agronomía: el pH controla la disponibilidad de todos los demás nutrientes (Weil and Brady, 2017).

2.4. Álgebra lineal

2.4.1. Vectorización

Una vez identificado el cultivo más compatible, la pregunta es: **¿cuánto le falta (o le sobra) al suelo para cumplir las necesidades de ese cultivo?**

El sistema responde representando tanto el estado del suelo como las necesidades del cultivo como **vectores en un espacio de 5 dimensiones:**

$$V = \left[N\ (kg/ha),\ P\ (kg/ha),\ K\ (kg/ha),\ pH,\ MO\ (\%)\right]$$

Estas 5 dimensiones capturan los factores más determinantes para la nutrición vegetal. Los tres primeros son los macronutrientes principales, ya en kg/ha (conversión aplicada). El pH y la materia orgánica se incluyen sin conversión porque son variables continuas en escala propia.

2.4.2. Vector de demanda del cultivo

La demanda de cada cultivo se calcula dinámicamente como el **promedio estadístico** de los registros de ese cultivo en el dataset de entrenamiento. Esto garantiza que la demanda refleja lo que realmente requieren las plantas según los datos observados (Mucherino et al., 2009):

```
# models/fertilizer_recommender_v2.py  __init__()

df_c = pd.read_csv(ruta_cultivos)
for cultivo, df_g in df_c.groupby("cultivo"):
    # Los promedios del dataset ya estan en mg/kg = se convierten a kg/ha
    self.demandas_cultivo[cultivo] = {
```

```
        "n":  float(df_g["N_mgkg"].mean() * 2.0),   # mg/kg a kg/ha
        "p":  float(df_g["P_mgkg"].mean() * 2.0),
        "k":  float(df_g["K_mgkg"].mean() * 2.0),
        "ph": float(df_g["ph"].mean()),              # sin conversion
        "mo": float(df_g["mo_pct"].mean()),          # sin conversion
    }
# Ejemplo: jitomate
# V_demanda = [180.0, 55.0, 190.0, 6.5, 3.2]
```

2.4.3. Vector de aporte del suelo

El CSV del agricultor (en mg/kg) se transforma en el vector de aporte del suelo con la misma conversión:

```
# models/fertilizer_recommender_v2.py  calcular_deficit()

v_s = np.array([
    float(propiedades_suelo["N_mgkg"]) * 2.0,   # 50 mg/kg -> 100 kg/ha
    float(propiedades_suelo["P_mgkg"]) * 2.0,   # 22 mg/kg -> 44 kg/ha
    float(propiedades_suelo["K_mgkg"]) * 2.0,   # 130 mg/kg -> 260 kg/ha
    float(propiedades_suelo["ph"]),              # 7.2
    float(propiedades_suelo["mo_pct"]),          # 1.8
], dtype=float)

# v_s = [100.0, 44.0, 260.0, 7.2, 1.8]
```

2.4.4. Operación: Resta de vectores

$$\Delta V = V_{demanda} - V_{suelo}$$

```
v_c = np.array([demanda["n"], demanda["p"], demanda["k"],
                demanda["ph"], demanda["mo"]], dtype=float)

delta_v = v_c - v_s
# delta_v = [180-100, 55-44, 190-260, 6.5-7.2, 3.2-1.8]
#         = [+80,    +11,   -70,     -0.7,     +1.4 ]
```

Cada componente del vector resultado tiene un significado directo:

$\Delta V[i] > 0 \rightarrow$ déficit: el suelo no tiene suficiente (hay que fertilizar).

$\Delta V[i] < 0 \rightarrow$ exceso: el suelo tiene más de lo que necesita el cultivo (no fertilizar).

$\Delta V[i] \cong 0 \rightarrow$ óptimo (dentro del margen de tolerancia).

Solo los componentes positivos (déficits) generan recomendación de fertilización:

```
# Solo N, P, K son corregibles con fertilizantes NPK
# pH y MO requieren cal/azufre o composta, fuera del alcance NPK
delta_aplicable = np.array([
    max(0.0, delta_v[0]),   # N: deficit de 80 kg/ha -> hay que aplicar
    max(0.0, delta_v[1]),   # P: deficit de 11 kg/ha -> hay que aplicar
    max(0.0, delta_v[2]),   # K: exceso (-70) -> no fertilizar con K
    0.0,                    # pH: no corregible con NPK
    0.0                     # MO: no corregible con NPK
], dtype=float)
```

Cuadro 3: Ejemplo completo con valores reales de requerimientos nutricionales (jitomate).

Parámetro	**Lab (mg/kg)**	**Aporte suelo (kg/ha)**	**Demanda jitomate (kg/ha)**	ΔV	**Estado**
N	50 mg/kg	100 kg/ha	180 kg/ha	$+80$	Déficit
P	22 mg/kg	44 kg/ha	55 kg/ha	$+11$	Déficit leve
K	130 mg/kg	260 kg/ha	190 kg/ha	-70	Exceso
pH	7.2	7.2	6.5	$-0{,}7$	Algo alcalino
MO	1.8 %	1.8 %	3.2 %	$+1{,}4$	Bajo

2.4.5. Clasificación del estado por umbrales agronómicos

Cada componente de ΔV se clasifica con umbrales calibrados para suelos mexicanos (Instituto Nacional de Investigaciones Forestales, Agrícolas y Pecuarias, 2019):

```
# models/fertilizer_recommender_v2.py

UMBRALES_ESTADO = {
    "N": {
        "exceso_toxico": 200,   # > 200 kg/ha sobre la demanda: danio toxico
        "exceso":         30,   # entre 30 y 200 kg/ha: exceso moderado
        "optimo_max":     10,   # margen de 10 kg/ha -> optimo
        "optimo_min":    -10,
```

```
        "critico":       -50,   # mas de 50 kg/ha bajo la demanda: critico
    },
    "P": { "exceso_toxico": 100, "exceso": 20,
           "optimo_max": 10, "optimo_min": -10, "critico": -40 },
    "K": { "exceso_toxico": 300, "exceso": 50,
           "optimo_max": 20, "optimo_min": -20, "critico": -80 },
    "pH": { "exceso_toxico": 1.5, "exceso": 0.5,
            "optimo_max": 0.3, "optimo_min": -0.3, "critico": -1.0 },
    "MO": { "exceso_toxico": 3.0, "exceso": 1.0,
            "optimo_max": 0.5, "optimo_min": -0.5, "critico": -1.5 },
}
```

Para el ejemplo anterior, $\Delta V = \pm 10\ kg/ha$ cae entre $10\ y\ 50\ kg/ha$: estado Déficit. Si fuera mayor de 50 sería Crítico.

2.5. Selección del fertilizante

2.5.1. Algoritmo greedy

Con el vector de déficit calculado, el sistema selecciona el fertilizante más eficiente para cada nutriente deficitario:

```
# models/fertilizer_recommender_v2.py  _seleccionar_fertilizantes_optimos()

for idx_nut, nutriente, valor_deficit in nutrientes_a_corregir:
    mejor_score = float("inf")
    for fert in catalogo:
        concentracion = fert["vector_npk"][idx_nut]
        if concentracion <= 0:
            continue  # este fertilizante no aporta el nutriente objetivo

        dosis_necesaria  = valor_deficit / concentracion
        costo_total_fert = dosis_necesaria * fert["precio_kg"]

        # Descartar si introduce exceso en nutriente ya sobreabundante
        tiene_conflicto = any(
            nut_idx != idx_nut
            and fert["vector_npk"][nut_idx] > 0
```

```
            and deficit_npk[nut_idx] <= 0
            for nut_idx in range(len(nutrientes))
        )
        if tiene_conflicto:
            continue

        score = costo_total_fert / (concentracion + 1)
        if score < mejor_score:
            mejor_score  = score
            mejor_opcion = {"fertilizante": fert, "dosis_kg_ha": dosis_necesaria}
```

La dosis resultante se expresa en costales de 50 kg para que sea accionable en campo:

```
def formatear_dosis_kg_ha(kg_ha: float) -> str:
    costales = kg_ha / 50.0
    return f"{kg_ha:.1f} kg/ha (Equivalente a {costales:.1f} costales de 50 kg)"
```

Con $\Delta N = 80\ kg/ha$ y Urea ($46\,\%\ N$): $dosis = 80\ /\ 0{,}46 = 173{,}9\ kg/ha \approx 3{,}5$ costales de 50 kg.

2.6. Cálculo diferencial

2.6.1. Optimización de dosis de fertilizante

Una vez que el sistema recomienda "aplica 3.5 costales de Urea", surge la pregunta natural: **¿qué pasa si aplico más?, ¿o menos?** El agricultor sabe por experiencia que un poco más de fertilizante no siempre da más cosecha, y demasiado puede quemar las raíces. Existe un punto óptimo. El módulo de cálculo diferencial lo encuentra y lo visualiza.

El sistema usa dos funciones que compiten:

1. **Función de rendimiento** (beneficio de fertilizar):

$$R(u) = a \cdot \ln(u + 1) + b$$

   - u = costales aplicados (eje X de la gráfica).
   - $\ln(u + 1)$ modela rendimientos marginales decrecientes: el primer costal tiene mucho efecto; el décimo, mucho menos.
   - a = tasa de respuesta (mayor cuando el déficit es alto).

- b = rendimiento base del suelo sin fertilización extra.

2. **Función de degradación** (costo de sobrefertilizar):

$$D(u) = c \cdot u^2$$

El crecimiento cuadrático penaliza las dosis excesivas, reflejando que el exceso acidifica el suelo, quema raíces y contamina el manto acuífero.

3. **Función de beneficio neto**:

$$B(u) = R(u) - D(u) = a \cdot \ln(u+1) + b - c \cdot u^2$$

2.6.2. El cálculo del punto óptimo

El punto óptimo u^* es donde agregar otro costal deja de valer la pena: donde la derivada de $B(u)$ es exactamente cero.

$$B'(u) = \frac{a}{u+1} - 2cu = 0$$

Despejando, esto se convierte en una ecuación cuadrática en u:

$$2cu^2 + 2cu - a = 0$$

Aplicando la fórmula general para ecuaciones cuadráticas (con coeficientes $2c$, $2c$, $-a$):

$$u^* = \frac{-2c + \sqrt{(2c)^2 + 4 \cdot 2c \cdot a}}{2 \cdot 2c} = \frac{-c + \sqrt{c^2 + 2ca}}{2c}$$

En código:

```
# app.py  api_fertilizer_optimization_for_crop()

# Parametros derivados dinamicamente del analisis de deficit real
a = max(1.0, 1.5 + deficit_total / 100.0)   # tasa de respuesta
b = max(1.0, 5.0 - exceso_total  / 50.0)    # rendimiento base

# c se calibra para que el optimo matematico coincida exactamente
# con la recomendacion vectorial previa (total_bags):
#   R'(u*) = a/(u*+1) - 2c*u* = 0  =>  c = a / (2*u**(u*+1))
```

```
c = a / (2 * total_bags * (total_bags + 1))

# Calculo por formula cuadratica
discriminante = c**2 + 2 * c * a
u_opt = (-c + np.sqrt(discriminante)) / (2 * c) if discriminante > 0 else 0.0

# Generacion de 200 puntos para las curvas
u = np.linspace(0, max(40, u_opt * 1.5), 200)
R = a * np.log(u + 1) + b    # curva azul: rendimiento
D = c * u ** 2               # curva roja: degradacion
B = R - D                    # curva verde: beneficio neto
```

2.6.3. Parametrización dinámica

Los parámetros a, b y c se recalculan para cada análisis; no son constantes globales:

- **El parámetro** a crece con el déficit total: un suelo muy pobre responde más dramáticamente al fertilizante.
- **El parámetro** b baja si hay excesos nutricionales: un suelo con K en exceso ya tiene un rendimiento base limitado por ese desbalance.
- **El parámetro** c se calibra para garantizar coherencia matemática: el pico de la curva de beneficio coincide exactamente con la dosis que ya recomendó el módulo vectorial.

Este diseño garantiza que la gráfica no es puramente decorativa: es la demostración matemática rigurosa de que la dosis recomendada es realmente la óptima.

2.6.4. Lectura de la gráfica

La gráfica se compone de tres curvas principales y un marcador de calibración:

- **Curva azul (**R**):** Sube rápido y se aplana. El primer costal tiene mucho impacto; el décimo, casi nulo.
- **Curva roja (**D**):** Casi plana al inicio, se acelera al final. El daño crece de forma cuadrática.
- **Curva verde (**$B = R - D$**):** Tiene forma de campana. La cresta representa el punto óptimo u^*.

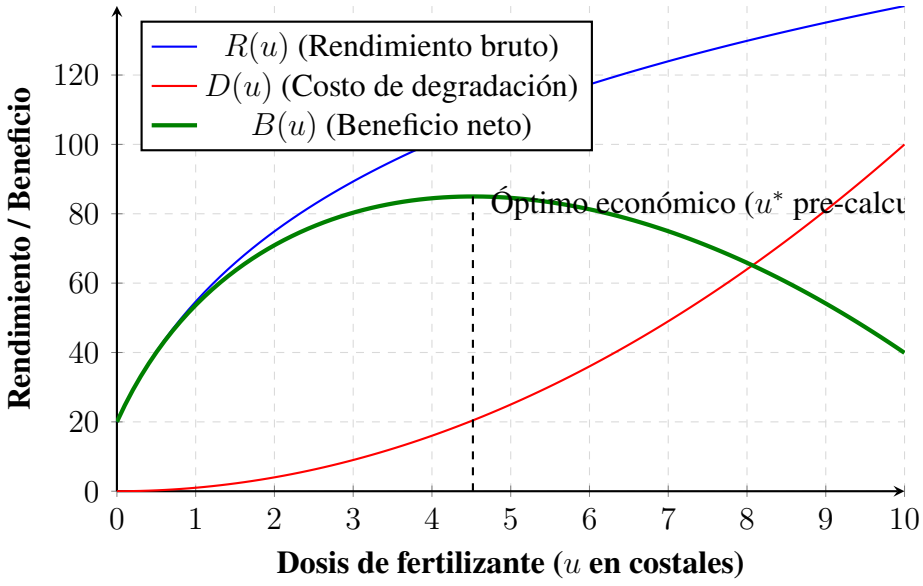


Figura 1: Dinámica de optimización: intersección entre el rendimiento marginal decreciente y la degradación cuadrática.

- **Punto negro (línea punteada):** La dosis recomendada por el modelo vectorial, la cual debe coincidir exactamente con la cresta de la campana.

Si el agricultor aplica menos de u^*, deja rendimiento sobre la mesa. Si aplica más, comienza a dañar el suelo sin obtener ningún beneficio adicional. Esta representación gráfica hace que la dinámica entre costo y beneficio sea fácil de entender sin necesidad de conocer los detalles matemáticos.

2.7. Estadística

2.7.1. Promedio de múltiples muestras por parcela

Un error frecuente en la agricultura es tomar una sola muestra por parcela y tratar ese valor como representativo de toda la superficie. Sin embargo, el suelo varía significativamente de un punto a otro dentro de la misma parcela (Weil and Brady, 2017).

Agroasis acepta múltiples muestras por parcela (tomadas en puntos distintos, por ejemplo en zigzag o en cuadrícula) y las promedia estadísticamente para obtener un perfil representativo.

2.7.2. Identificación automática de parcelas

El sistema detecta automáticamente si el CSV tiene una o varias parcelas. Si existe la columna `id_parcela`, agrupa las filas por prefijo (reconociendo que `ParcelaA_m1` y `ParcelaA_m2` son

muestras de la misma parcela A):

```
# app.py

def _prefijo_parcela(nombre: str) -> str:
    """Extrae el identificador eliminando el sufijo de muestra.
    Ejemplos:
      'ParcelaA_m1' -> 'ParcelaA'
      'norte-muestra2' -> 'norte'
      'P1' -> 'P1' (sin sufijo -> nombre completo)
    """
    sufijo = re.sub(
        r'[\s_\-]*(m|muestra|s|sample|r)?\s*\d+\s*$',
        '', str(nombre), flags=re.IGNORECASE
    ).strip()
    return sufijo if sufijo else str(nombre)
```

2.7.3. Cálculo de media aritmética

Una vez agrupadas las muestras, se calcula la media aritmética de cada parámetro numérico:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

```
# app.py

datos_prom = {
    col: float(df_grupo[col].mean())
    for col in SOIL_FEATURES
    if col in df_grupo.columns
}

# Textura: variable categorica -> moda (valor mas frecuente)
if "textura_suelo" in df_grupo.columns:
    datos_prom["textura_suelo"] = df_grupo["textura_suelo"].mode()[0]
```

Ejemplo con tres muestras. Parcela de 2 ha, tres muestras en puntos distintos:

El perfil promedio se convierte a kg/ha:

Cuadro 4: Resultados de las muestras individuales y cálculo del promedio.

Muestra	**N (mg/kg)**	**P (mg/kg)**	**K (mg/kg)**	**pH**
m1 (esquina norte)	38	12	95	6.9
m2 (centro)	51	18	110	7.1
m3 (esquina sur)	44	15	105	6.8
Promedio	**44.3**	**15.0**	**103.3**	**6.93**

Cuadro 5: Conversión del vector promedio a unidades de campo.

Parámetro	$\bar{X}$ **(mg/kg)**	$\bar{X} \times 2{,}0$ **(kg/ha)**
N	44.3	88.7
P	15.0	30.0
K	103.3	206.7

Este vector promedio es el que alimenta al clasificador y al módulo vectorial.

2.7.4. Estadísticas descriptivas adicionales

El sistema también calcula estadísticas para evaluar la homogeneidad de la parcela:

```
# app.py  _analizar_parcela_stats()
media = float(np.mean(datos))
std   = float(np.std(datos))
resultados[feature] = {
    "media": media,
    "std":   std,
    "min":   float(np.min(datos)),
    "max":   float(np.max(datos)),
    "cv":    float(std / media * 100) if media != 0 else 0.0,
}
```

El Coeficiente de Variación ($CV = \frac{\sigma}{\bar{X}} \times 100\,\%$) es el indicador clave: si el nitrógeno tiene $CV > 30\,\%$, la parcela es heterogénea y podría justificarse una fertilización diferencial por zonas. Si el CV es bajo ($< 15\,\%$), una sola recomendación representa bien toda la parcela.

Matriz de salud visual para múltiples parcelas. Cuando el CSV contiene varias parcelas distintas, cada una recibe un *score* de salud de 0 a 100 calculado comparando los promedios de N, P y K contra los límites de referencia agronómica:

```
def _calcular_score_salud(promedios):
```

```
    scores = []
    for nutriente, limites in NUTRIENT_LIMITS.items():
        if nutriente in promedios:
            v = promedios[nutriente]
            if v >= limites["alto"]:    scores.append(100)
            elif v >= limites["medio"]: scores.append(75)
            elif v >= limites["bajo"]:  scores.append(50)
            else:                       scores.append(25)
    return float(np.mean(scores)) if scores else 50.0
```

El *score* se mapea a un color para la visualización:

Cuadro 6: Mapeo del *score* de salud a estados visuales.

Score	Estado	Color (Hexadecimal)
65–100	Excelente	Verde (`#27ae60`)
50–65	Bueno	Verde claro (`#9acd32`)
40–50	Aceptable	Naranja (`#f39c12`)
30–40	Pobre	Naranja oscuro (`#e67e22`)
0–30	Muy pobre	Rojo (`#e74c3c`)

2.7.5. Prueba de hipótesis para desequilibrios nutricionales

Cuando el análisis vectorial detecta un desequilibrio (déficit o exceso), surge la pregunta técnica obligada: ¿es real o es solo ruido estadístico? Para responder, el sistema implementa una prueba de hipótesis en JavaScript, en el *frontend*, sobre el parámetro con el desequilibrio más extremo (mayor $|\Delta V|$). Esta validación solo se ejecuta si existen múltiples muestras ($n \geq 2$).

Cálculo en JavaScript: El sistema calcula la varianza muestral, el error estándar y el estadístico t. Si $n < 30$, utiliza una tabla t-Student integrada; si $n \geq 30$, emplea la distribución Z Normal (valor crítico de $1{,}645$).

```
// Se evalua solo el parametro N, P o K con mayor |DeltaV|
const t_calc = se > 0 ? (mean - mu0) / se : 0;
const t_crit = tCritico(n - 1);
const rechazaH0 = esDeficit ? (t_calc < -t_crit) : (t_calc > t_crit);
```

Interpretación:

Se rechaza H_0: El déficit o exceso es estadísticamente significativo. La intervención correctiva es una necesidad agronómica comprobada.

No se rechaza H_0: La anomalía podría deberse a la variabilidad natural del terreno (ruido estadístico), pero se recomienda una fertilización preventiva de mantenimiento para evitar la degradación futura del perfil edáfico.

2.8. Evolución del suelo sin reposición

2.8.1. El problema de la sostenibilidad

Un suelo puede estar en estado óptimo hoy, pero si el agricultor cosecha temporada tras temporada sin reponer los nutrientes, el balance irá deteriorándose progresivamente. Este módulo simula ese agotamiento para los próximos 5 ciclos de cultivo y lo presenta en una tabla y gráfica interactiva.

El modelo de agotamiento. El modelo es iterativo: para cada temporada, el aporte actual del suelo se reduce según lo que el cultivo extrae:

$$\text{aporte}_t = \text{aporte}_{t-1} - (\text{demanda}_{\text{cultivo}} \times \text{FACTOR_EXTRACCION})$$

Donde FACTOR_EXTRACCION $= 0{,}65$ representa la fracción de nutrientes que el cultivo extrae de forma neta. La diferencia con 1.0 (el 35 % restante) es lo que vuelve al suelo vía raíces, residuos y biomasa microbiana.

Para el pH y la materia orgánica (MO), la degradación es autónoma; no depende de la demanda del cultivo, sino del proceso natural de acidificación y mineralización bajo cultivo intensivo (Obalum et al., 2017):

$$\text{pH}_t = \text{pH}_{t-1} - 0{,}08 \quad \text{(acidificación: 0.08 unidades por temporada)}$$

$$\text{MO}_t = \text{MO}_{t-1} - 0{,}12 \quad \text{(pérdida de MO: 0.12 \% por temporada)}$$

2.8.2. Inferencia para casos sin datos

Gran parte de los pequeños productores en México no tienen acceso rápido o económico a análisis de laboratorio de suelos. Para evitar que el sistema sea exclusivo, Agroasis agrega un modo manual que estima las propiedades del suelo basándose en observaciones del agricultor.

Lógica de inferencia. En lugar de pedir mediciones exactas en mg/kg, el sistema hace preguntas sobre el suelo (por ejemplo, textura, historial de cultivos, precipitación y altitud). Cada respuesta ajusta el análisis inferido:

Textura: Define la retención de agua y nutrientes base.

Historial de cultivos: Si se plantó maíz antes (altamente extractivo de N), se infiere una penalización en el nitrógeno disponible. Si se plantó frijol (fijador de N), se asume un bono de nitrógeno.

Clima y altitud: Ajustan la velocidad de mineralización de la materia orgánica y el lavado de nutrientes por lluvias.

Resultado aproximado. A partir de las respuestas, el *backend* (servidor de la aplicación) construye un vector de suelo aproximado que alimenta al mismo clasificador y módulo vectorial que se usa con datos de laboratorio. La diferencia es que los valores inferidos tienen mayor incertidumbre, por lo que el sistema lo indica explícitamente al usuario para orientar sus decisiones de manera responsable.

3. Marco teórico

3.1. Agronomía y propiedades físico-químicas del suelo

La productividad agrícola depende en gran medida de las propiedades físico-químicas del suelo, ya que estas determinan la disponibilidad de nutrientes, la retención de agua y el desarrollo de los cultivos. El suelo constituye un sistema dinámico donde interactúan componentes minerales, materia orgánica, microorganismos y factores ambientales como la humedad y el pH, los cuales influyen directamente en el crecimiento vegetal (Weil and Brady, 2017).

Dentro de las variables más relevantes para evaluar la fertilidad del suelo se encuentran el potencial de hidrógeno (pH), la conductividad eléctrica, la textura y la concentración de macro y micronutrientes. Estas características permiten aproximar la capacidad de producción de un terreno y facilitan la toma de decisiones relacionadas con fertilización, selección de cultivos y manejo agronómico.

3.1.1. Nutrición vegetal

La nutrición vegetal se refiere a la forma en que las plantas absorben, transportan y utilizan los nutrientes esenciales para su desarrollo. Para que un cultivo alcance una producción adecuada y eficiente no basta únicamente con la disponibilidad de agua y radiación solar, sino que requiere cantidades equilibradas de nutrientes minerales presentes en el suelo (Weil and Brady, 2017).

Los nutrientes requeridos por las plantas se clasifican en macronutrientes y micronutrientes de acuerdo con la cantidad necesaria para el desarrollo vegetal. Los macronutrientes primarios incluyen

nitrógeno (N), fósforo (P) y potasio (K), considerados esenciales debido a su participación en procesos metabólicos fundamentales: el nitrógeno interviene en la síntesis de proteínas y clorofila; el fósforo participa en la transferencia de energía mediante ATP; y el potasio regula procesos osmóticos y el balance hídrico de la planta. Por otro lado, los micronutrientes, como hierro, zinc, manganeso, cobre y boro, son requeridos en menores cantidades, aunque siguen siendo indispensables para múltiples procesos enzimáticos y fisiológicos.

3.1.2. pH y conductividad eléctrica

El potencial de hidrógeno (pH) representa el nivel de acidez o alcalinidad del suelo y constituye uno de los indicadores más importantes en agricultura, debido a su influencia sobre la disponibilidad de nutrientes. La mayoría de los cultivos presentan un desarrollo óptimo en rangos de pH entre 5.5 y 7.0, donde la absorción de nutrientes esenciales alcanza niveles adecuados. Valores fuera de este intervalo pueden provocar toxicidad o deficiencias nutricionales (Weil and Brady, 2017).

Asimismo, la conductividad eléctrica permite estimar la concentración de sales solubles presentes en el suelo. Niveles elevados de salinidad afectan la absorción de agua por parte de las plantas y pueden disminuir significativamente el rendimiento agrícola.

3.2. Agricultura de precisión

La agricultura de precisión surge como una estrategia tecnológica orientada a optimizar la producción agrícola mediante el análisis de datos provenientes del suelo, el clima y el rendimiento de cultivos (Gebbers and Adamchuk, 2010). Este enfoque utiliza herramientas computacionales y modelos de análisis de datos para identificar patrones espaciales y temporales dentro de los sistemas agrícolas. Su objetivo principal consiste en mejorar la eficiencia del uso de recursos, reducir costos de producción y aumentar la sostenibilidad ambiental.

3.3. Machine learning

El machine learning constituye una rama de la inteligencia artificial enfocada en el desarrollo de algoritmos capaces de identificar patrones y realizar predicciones a partir de datos históricos (Mucherino et al., 2009). En el ámbito agrícola, estas técnicas han sido utilizadas para clasificación de cultivos, predicción de rendimientos, análisis de fertilidad del suelo y detección de enfermedades vegetales. Modelos como Random Forest y Gradient Boosting han mostrado alta capacidad para trabajar con variables ambientales y agrícolas debido a su precisión y capacidad para manejar relaciones no lineales entre datos (Sharma et al., 2021).

La aplicación de algoritmos predictivos en agricultura permite generar sistemas de apoyo a la toma de decisiones basados en evidencia estadística y análisis automatizado de información.

3.4. Fundamentos matemáticos

3.4.1. Modelos de predicción y reconocimiento de patrones

El reconocimiento de patrones constituye una disciplina orientada a identificar relaciones y similitudes dentro de conjuntos de datos complejos. En el contexto agrícola, estas técnicas permiten analizar características del suelo y relacionarlas con determinados tipos de cultivo o niveles de productividad. Los modelos predictivos basados en aprendizaje automático utilizan información histórica para generar estimaciones y clasificaciones a partir de múltiples variables simultáneamente (Mucherino et al., 2009). Este enfoque resulta especialmente útil en agricultura debido a la naturaleza multivariable de los sistemas de producción, donde intervienen factores químicos, físicos y ambientales.

3.4.2. Similitud de datos

El análisis multivariable permite estudiar simultáneamente múltiples características de un sistema agrícola, considerando la interacción existente entre nutrientes, propiedades del suelo y variables ambientales. Dentro de la ciencia de datos, las medidas de similitud y distancia son utilizadas para comparar observaciones y determinar qué tan cercanas son entre sí respecto a determinadas características. Estas técnicas han sido ampliamente utilizadas en estudios de clasificación agrícola y análisis de fertilidad (Sharma et al., 2021).

3.4.3. Sistemas de apoyo para la toma de decisiones agrícolas

Los sistemas de apoyo a la toma de decisiones (*Decision Support Systems*, DSS) son herramientas computacionales diseñadas para asistir en la resolución de problemas complejos mediante el análisis de información y generación de recomendaciones (Mucherino et al., 2009). En el sector agrícola, los DSS han sido utilizados para optimizar procesos relacionados con fertilización, selección de cultivos, manejo hídrico y planificación de siembra. Estos sistemas integran bases de datos, modelos predictivos y análisis estadísticos para mejorar la eficiencia productiva y reducir la incertidumbre en la toma de decisiones.

4. Limitaciones y prospectivas

4.1. Limitaciones

4.1.1. La limitante de datos agronómicos en México

Durante el desarrollo del proyecto Agroasis se encontró el obstáculo de la **ausencia de bases de datos centralizadas, estandarizadas y de acceso público**. Si bien instituciones como el INIFAP (Instituto Nacional de Investigaciones Forestales, Agrícolas y Pecuarias), el SIAP (Servicio de Información Agroalimentaria y Pesquera) y diversas universidades generan conocimiento sobre los cultivos y sus nutrientes (Servicio de Información Agroalimentaria y Pesquera, 2022), esta información se encuentra típicamente fragmentada en:

- Folletos técnicos regionales en formato PDF.
- Tesis académicas y estudios de caso muy localizados.
- Anuarios estadísticos de rendimiento que omiten las variables químicas del suelo.

Para entrenar un modelo como Random Forest se requería un dataset (base de datos) tabular en formato CSV estructurado que contuviera muestras de suelo con múltiples variables químicas y físicas (N, P, K, pH, materia orgánica, micronutrientes, textura) emparejadas con las variedades de cultivos óptimos. Para solucionar esto, se diseñó una metodología de recopilación, digitalización y estructuración de casos representativos para construir una base de conocimientos propia: `cultivos/mexico/v3/final.csv`.

4.2. Metodología de construcción de la base de datos

Para asegurar que el modelo de machine learning ofrezca recomendaciones coherentes con la realidad agrícola mexicana, la creación del dataset se alejó de promedios teóricos genéricos y se basó en una recopilación metódica de perfiles agrícolas regionales. El proceso constó de tres fases principales.

Búsqueda y selección de fuentes documentales

El primer paso consistió en rastrear la literatura técnica agrícola del país para identificar registros y recomendaciones de parcelas de alto rendimiento. Se consultaron manuales de producción y folletos técnicos del INIFAP orientados a diferentes zonas geográficas (Instituto Nacional de Investigaciones Forestales, Agrícolas y Pecuarias, 2019). El objetivo de esta etapa fue identificar

cómo cambian los requerimientos de un mismo cultivo dependiendo de su entorno. Por ejemplo, documentar que el maíz del sur del país enfrenta condiciones del suelo diametralmente opuestas al del norte:

1. **Noroeste (Sinaloa/Sonora):** Manejo de alta tecnología. Suelos típicamente alcalinos (pH 7.2–7.5), baja materia orgánica ($< 1{,}5\,\%$), alta retención de bases (Ca, Mg).
2. **Sureste (Oaxaca/Chiapas):** Sistemas de temporal. Suelos ligeramente ácidos (pH 5.5–6.0), mayor materia orgánica (2.5–3.0 %), afectados por alta precipitación.
3. **Bajío (Guanajuato/Jalisco):** Suelos francos, pH neutro a ligeramente ácido, régimen hídrico mixto.

Ejemplo de mapeo tangible de fuentes consultadas. A continuación se ilustra una fracción de las referencias técnicas que sirvieron como "ancla" para estructurar los registros regionales dentro del archivo CSV:

Cuadro 7: Referencias técnicas del INIFAP utilizadas como valores ancla.

Fuente / Documentación técnica (INIFAP)	**Cultivos mapeados**	**Variables extraídas**
Guía técnica para la producción de maíz en el norte de Sinaloa	Maíz grano blanco, maíz amarillo	N, P, K (bray1), textura franco-limosa, pH alcalino, clima
Manual de producción de frijol en el estado de Zacatecas	Frijol pinto, frijol guajillo	N, P (olsen), K, precipitaciones, altitud
Manejo agronómico del cultivo de aguacate Hass en Michoacán	Aguacate Hass, fuerte	pH ligeramente ácido, MO alta (>3.5 %), altitud, temperatura
Paquete tecnológico para jitomate en invernadero (Bajío)	Jitomate bola, saladette	Macronutrientes altos, CE, micronutrientes (Fe, Zn)

Digitalización y estandarización de casos de estudio

Con la información técnica recabada, el principal reto fue la homologación de los datos. Los estudios agronómicos a menudo reportan los nutrientes en diferentes unidades o escalas dependiendo del laboratorio que realizó el análisis. Se procedió a la digitalización manual y extracción de estos perfiles en un formato tabular estricto. Cada **caso de estudio** regional se tradujo en un registro completo dentro del *dataset*, asegurando que:

1. Todos los macronutrientes y micronutrientes quedaran expresados bajo una misma escala (mg/kg).

2. Se respetaran las relaciones biológicas documentadas (por ejemplo, registrar las deficiencias naturales de micronutrientes como hierro y zinc en perfiles con pH alcalino).

3. Se categorizara adecuadamente la textura del suelo para su posterior codificación algorítmica.

Esta estandarización permitió transformar información cualitativa y fragmentada en vectores numéricos precisos listos para el entrenamiento.

4.2.1. Cambio de escala (mg/kg a kg/ha)

Durante la estructuración del *dataset* y el diseño del algoritmo surgió una pregunta: si la base de datos y los resultados de los laboratorios agrícolas están estandarizados en mg/kg, ¿por qué el sistema Agroasis realiza todos sus cálculos internos en kg/ha?

La respuesta es que las plantas consumen "masa" (kg/ha), no "concentración" (mg/kg). Es físicamente imposible calcular cuántos costales de 50 kg de fertilizante requiere una parcela usando únicamente medidas de concentración. Para saber el déficit real, hay que transformar esa concentración a la masa total disponible en una hectárea.

Por lo tanto, la decisión fue crear el *dataset* almacenando la información tal como sale del laboratorio (mg/kg), dejando al programa la transformación invisible de esta concentración hacia la masa total disponible (kg/ha) justo antes de alimentar a los modelos predictivos.

Integración de variables bioclimáticas

Para dar mayor robustez al clasificador y evitar que el modelo recomendara cultivos incompatibles con el entorno, el *dataset* se enriqueció asociando a cada perfil de suelo las variables climáticas típicas de su región de origen:

1. Precipitación anual (mm).

2. Temperatura media (°C).

3. Altitud (msnm).

Estas variables actúan como "filtros contextuales" en el modelo de Random Forest. De esta forma, si el sistema procesa un suelo químicamente apto para papaya, pero con una altitud asociada de 2,500 msnm (clima frío), el modelo aprenderá a descartar el cultivo tropical y priorizar opciones viables para esa elevación.

4.2.2. Estructura final del dataset

El resultado se muestra en el archivo `cultivos_mexico_v3_final.csv`. El *dataset* cuenta con 24 columnas que representan el perfil completo de cada observación:

- **Identificadores contextuales:** `cultivo`, `variedad`, `region`, `estado_ref`
- **Macronutrientes (mg/kg):** `N_mgkg`, `P_mgkg`, `K_mgkg`, `Ca_mgkg`, `Mg_mgkg`, `S_mgkg`
- **Micronutrientes (mg/kg):** `Fe_mgkg`, `Zn_mgkg`, `Mn_mgkg`, `Cu_mgkg`, `B_mgkg`
- **Propiedades físico-químicas:** `ph`, `mo_pct` (materia orgánica), `ce_ds_m` (conductividad eléctrica), `textura_suelo`
- **Clima y geografía:** `precipitacion_mm`, `temp_media_c`, `altitud_msnm`
- **Desempeño esperado:** `rendimiento_ton_ha`

4.3. Prospectivas

Agroasis nació como respuesta a una necesidad real y concreta: el productor agrícola mexicano, ya sea pequeño, mediano o técnico agropecuario, que recibe un análisis de laboratorio de su parcela no siempre cuenta con los conocimientos técnicos para interpretarlo y traducirlo en acciones específicas. En ese sentido, la trayectoria natural del proyecto apunta a consolidar y ampliar esa accesibilidad, sin desvirtuar su enfoque original: ser la herramienta de referencia para cualquier productor que ya cuenta con una muestra analizada de su suelo.

El principal paso a corto plazo consiste en eliminar la restricción del formato CSV como único canal de entrada de datos. Actualmente, muchos laboratorios de análisis de suelos en México entregan sus resultados en formatos variados: reportes en PDF, hojas de cálculo Excel, archivos JSON generados por sus propios sistemas o incluso documentos escaneados. Requerir que el usuario transforme esos documentos a CSV antes de subirlos representa una barrera técnica que excluye a productores con menor familiaridad digital. El siguiente objetivo de desarrollo de Agroasis contempla la incorporación de un módulo de lectura universal que procese archivos PDF estructurados mediante extracción de texto, hojas de cálculo Excel con la librería `openpyxl`, archivos JSON con la librería nativa de Python, y, en una etapa posterior, reportes escaneados mediante reconocimiento óptico de caracteres (OCR) con herramientas como Tesseract. Esta evolución convertiría a Agroasis en una herramienta verdaderamente accesible para el ecosistema de laboratorios agrícolas nacionales.

En materia de costos, si el sistema se pusiera en producción en su estado actual, los gastos operativos serían modestos: el alojamiento del servidor (backend), por ejemplo, en una instancia de bajo costo en plataformas como Railway o Render, tendría un costo estimado de entre 10 y 25 USD mensuales para un volumen moderado de consultas. El modelo de Random Forest, al estar serializado con `pickle`, no requiere reentrenamiento frecuente; sin embargo, actualizar el dataset y reentrenar el clasificador semestralmente para incorporar nuevos perfiles regionales implicaría aproximadamente cuatro a ocho horas de trabajo técnico por ciclo (Breiman, 2001). El costo de mantenimiento del catálogo de cultivos, revisión agronómica y validación de los vectores de demanda nutricional, podría cubrirse mediante la colaboración con instituciones como el INIFAP (Instituto Nacional de Investigaciones Forestales, Agrícolas y Pecuarias, 2019) o con estudiantes de agronomía en servicio social, lo que mantendría los gastos operativos dentro de un rango accesible para un proyecto académico en transición hacia la práctica real.

A mediano plazo, el proyecto contempla ampliar el catálogo de cultivos más allá de los registros actuales, incorporando variedades de las principales zonas productoras de los 32 estados del país y añadiendo cultivos emergentes como amaranto, chía y quinoa, cuya demanda ha crecido notablemente en los últimos años. Asimismo, se prevé la vinculación con laboratorios de análisis de suelos y con programas de extensionismo agrícola de la Secretaría de Agricultura y Desarrollo Rural (SADER) y el INIFAP (Secretaría de Agricultura y Desarrollo Rural, 2020), de manera que Agroasis pueda ser adoptado como herramienta de apoyo en las cadenas de diagnóstico ya existentes, llegando directamente a quienes más lo necesitan: los productores de pequeña y mediana escala.

Finalmente, desde el punto de vista de la investigación académica, el sistema abre líneas de trabajo como la comparación sistemática de algoritmos de clasificación, Gradient Boosting, Support Vector Machines, redes neuronales, sobre el dataset construido, el análisis de la importancia de variables por región geográfica, y el estudio del impacto real de las recomendaciones del sistema sobre el rendimiento de parcelas piloto (Sharma et al., 2021). Este último punto permitiría validar empíricamente el modelo con datos de campo reales, cerrando el ciclo entre el desarrollo tecnológico y la práctica agrícola cotidiana.

5. Conclusión

Agroasis proporciona un enfoque tecnológico que facilita la selección adecuada de cultivos y optimiza el uso de fertilizantes en las parcelas agrícolas, mediante la gestión y el análisis de datos que permiten tomar decisiones informadas y sostenibles para el manejo del suelo.

Gracias a la utilización de técnicas de aprendizaje automático como Random Forest (Breiman, 2001), junto con métodos matemáticos sólidos, el sistema predice con gran precisión los cultivos más aptos en función de las condiciones específicas de cada parcela. El análisis vectorial permite detectar de manera precisa las carencias y los excesos de nutrientes, lo que facilita una aplicación óptima de fertilizante evitando la sobrefertilización.

El módulo de optimización diferencial proporciona un valor adicional significativo al sistema, al establecer la cantidad ideal de fertilizante que maximiza la producción al tiempo que minimiza gastos y daños. Asimismo, la capacidad de analizar múltiples muestras por área y crear un perfil representativo permite una evaluación más exacta y una gestión ajustada a la variabilidad del terreno.

El uso de la trazabilidad y la simulación del agotamiento del suelo fortalece el manejo integral del terreno, permitiendo una visión dinámica del estado del suelo a lo largo del tiempo, fundamental para adoptar prácticas que garanticen la sostenibilidad y la conservación de este recurso (Obalum et al., 2017). Este enfoque integral no solo facilita decisiones informadas y estratégicas, sino que también impulsa la transición hacia una agricultura más innovadora, responsable y resiliente.

Este sistema no solo mejora la rentabilidad del agricultor al reducir costos por uso innecesario de insumos, sino que también promueve prácticas agrícolas sostenibles, mitigando la degradación del suelo y su impacto ambiental. La incorporación de vectores multidimensionales para evaluar el déficit nutricional del suelo y la optimización diferencial del fertilizante ilustran cómo la tecnología puede traducir datos complejos en decisiones accesibles y efectivas.

Por lo tanto, Agroasis se consolida como una solución que traduce datos complejos de laboratorio en decisiones claras, accesibles y viables para el día a día en el campo. Al integrar el cálculo vectorial de nutrientes, los costos estimados y la proyección del agotamiento del suelo a futuro, el productor cuenta con una herramienta integral para anticipar problemas y cuidar su inversión. Con este enfoque, Agroasis demuestra que la tecnología aplicada a la agronomía es el camino para modernizar el sector, asegurando un equilibrio real entre la productividad de las parcelas y la conservación de la tierra (Gebbers and Adamchuk, 2010).

Referencias

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

Food and Agriculture Organization of the United Nations (2015). Status of the world's soil resources. Technical report, FAO, Rome, Italia.

Gebbers, R. and Adamchuk, V. I. (2010). Precision agriculture and food security. *Science*, 327(5967):828–831.

Instituto Nacional de Investigaciones Forestales, Agrícolas y Pecuarias (2019). Guía técnica para la producción de maíz en el norte de sinaloa. Technical report, INIFAP, Culiacán, México.

Mucherino, A., Papajorgji, P., and Pardalos, P. M. (2009). *Data Mining in Agriculture*. Springer, New York, NY.

Obalum, S. E., Chibuike, G. U., Peth, S., and Ouyang, Y. (2017). Soil organic matter as sole indicator of soil degradation. *Environmental Monitoring and Assessment*, 189(4):176.

Secretaría de Agricultura y Desarrollo Rural (2020). Panorama agroalimentario 2020. Technical report, SADER, Ciudad de México, México.

Servicio de Información Agroalimentaria y Pesquera (2022). Anuario estadístico de la producción agrícola. Technical report, SIAP, Ciudad de México, México.

Sharma, A., Jain, A., Gupta, P., and Chowdary, V. (2021). Machine learning applications for precision agriculture: A comprehensive review. *IEEE Access*, 9:4843–4873.

Weil, R. R. and Brady, N. C. (2017). *The Nature and Properties of Soils*. Pearson, Columbus, OH, 15th edition.